

Lab 1 Report

CSE 60685 Machine Learning for Embedded Systems Image Classification and Inference Latency on Raspberry Pi

On the supplied Raspberry Pi 4 Model B Rev. 1.5 running 64-bit Raspberry Pi OS (Debian 13 trixie, release 13.5) and Python 3.13.5, I measured `samples/chelsea.png` with one CPU thread, 10 untimed warm-ups and 100 timed calls per phase, using the unchanged `benchmark.py`.

ONNX Runtime 1.23.2 used `CPUExecutionProvider` with FP32, batch-one inputs. Runs were sequential with the same CanaKit power supply, open case lid, running fan and ondemand CPU governor. The two required runs began at 43.8–45.3 °C before warm-up; sampled temperatures across all runs stayed at or below 51.1 °C, with all recorded power/throttling flags equal to `0x0`.

Inference latency results

Model	ONNX file size (MiB)	Median inference latency (ms)
MobileNetV3-Small	9.71	39.351
SqueezeNet 1.1	4.73	90.091

1. Which model is faster, and does file size predict latency?

MobileNetV3-Small was 2.29 times faster, although its model file is larger. SqueezeNet’s smaller file therefore did not imply lower latency. Parameter count and file size describe stored weights; runtime also depends on feature-map sizes, arithmetic, memory traffic and CPU kernel efficiency. Static inspection of the supplied ONNX graphs gives approximately 54.9 million convolution MACs for MobileNet versus 349.2 million for SqueezeNet (Conv nodes only). MobileNet includes 11 grouped/depthwise convolutions, while SqueezeNet uses dense convolutions and concatenations. These differences help explain why weight size alone is insufficient; aggregate timings do not isolate each operation’s contribution.

2. Timer placement and measurement boundary

In `time_inference` in `latency.py`, I placed the counters immediately around the model call:

```
start = perf_counter_ns()
session.run(output_names, feed)
end = perf_counter_ns()
return (end - start) / 1_000_000
```

The result is in milliseconds and includes the synchronous CPU model call and Python call overhead. Model/session loading, image decoding and preprocessing occur outside the timer because the goal is repeated inference on a prepared tensor, rather than startup or the entire pipeline. Softmax, printing and file output are also excluded.

Optional exploration and validation

With four threads, inference medians were 21.702 ms (MobileNet) and 42.988 ms (SqueezeNet), giving one-to-four-thread speedups of $1.81\times$ and $2.10\times$. The environment check passed for both models. MobileNet printed five Chelsea predictions; the top result was tiger cat (49.31%). The timer exercise completed 10 warm-ups and 100 timed calls. The camera was listed as `ov5647`; `camera_check.py` reported PASS for a readable 640×480 JPEG, which was also classified. Each benchmark CSV retains all 200 measurements: 100 inference-only and 100 image-to-result calls. Code, JSON/CSV results and the camera image are preserved.